**Подготовка к защите — простыми словами**

Эта памятка относится именно к нашей программе. В ней есть короткие ответы для защиты и объяснения, чтобы понимать, что ты говоришь. Все вопросы преподавателя предсказать невозможно, но основные темы задания здесь разобраны.

## 1. Что рассказать о работе за одну минуту

«В программе выполнены два варианта: выделение контуров и удаление шума усредняющими фильтрами. Для контуров используются Лаплас, Робертс и Собел. Для удаления шума — обычное среднее и гауссов фильтр. По заданию двух вариантов достаточно для оценки 3.

Для хранения изображения используется класс Image. Он умеет читать и сохранять PNG, получать и менять пиксели. Память освобождается автоматически с помощью умного указателя. Сами фильтры находятся в отдельном файле, потому что по заданию нельзя добавлять новые методы в Image.

Я сравниваю результаты на двух картинках: с крупными фигурами и с мелкими деталями. Из трёх проверенных усредняющих настроек меньшую ошибку дал гауссов фильтр 5×5. Но на мелких деталях размытие может навредить сильнее, чем помогает удаление шума. В программе есть тесты: прошли 18 тестов и 323 проверки».

Не обязательно повторять текст слово в слово. Главное — понимать каждое предложение и уметь показать соответствующее место в коде.

## 2. Что нажимать и показывать преподавателю

1. Открой Lab.sln в Visual Studio 2022. Сверху выбери Debug и x64.

2. Нажми правой кнопкой по проекту Lab → «Назначить запускаемым проектом». Нажми Ctrl+F5.

3. Программа обработает test_input/shapes_noisy.png. Результаты окажутся в test_output/shapes_noisy. Картинки нужно открыть самостоятельно: программа не показывает их отдельным окном.

4. Сначала покажи чистую картинку test_input/shapes.png и её шумную версию test_input/shapes_noisy.png. Затем результаты фильтров.

5. Для контуров сравни laplace.png, roberts.png и sobel.png. Чистые контуры находятся в test_output/shapes, контуры с шумом — в test_output/shapes_noisy.

6. Для удаления шума сравни mean3.png, mean5.png и gauss5.png.

7. Покажи такие же результаты из test_output/detail_noisy. На этой картинке лучше видна потеря тонких линий.

8. Назначь запускаемым проект Tests и нажми Ctrl+F5. Покажи, что все тесты прошли.

9. Открой отчёт с формулами и результатами. Будь готов открыть Image.cpp и Filters.cpp.

Для нового входного файла: свойства проекта Lab → «Отладка» → «Аргументы команды». Например: test_input/detail_noisy.png test_output/detail_noisy. Первый путь — откуда брать картинку, второй — куда сохранять результат. Рабочий каталог уже настроен на папку с Lab.sln.

Важно: перед защитой проверь запуск на своём компьютере. В облаке программа проверялась под Linux; работа Visual Studio на твоём Windows-компьютере требует отдельной проверки.

### 3. Где что находится

| **Файл** | **Простое объяснение** |
|---|---|
| Image.h | Перечень того, что умеет изображение, и какие данные оно хранит |
| Image.cpp | Как изображение читает PNG, хранит пиксели, копируется и сохраняется |
| Filters.h | Объявления toGrayscale, applySobel, applyRoberts, applyLaplacian, applyBoxBlur, applyGaussianBlur |
| Filters.cpp | Сами расчёты Лапласа, Робертса, Собела и остальных фильтров |
| Source.cpp | С чего начинается программа и в каком порядке всё запускается |
| tests_image.cpp | Проверки Image, которые были в задании |

| Файл | Простое объяснение |
|---|---|
| tests_filters.cpp | Дополнительные проверки фильтров и работы с памятью |

Source.cpp запускает функции Filters. Их названия соответствуют методам: applySobel — Собел, applyRoberts — Робертс, applyLaplacian — Лаплас, applyBoxBlur — обычное среднее, applyGaussianBlur — Гаусс. Сначала toGrayscale получает серый вход.

В одном запуске программа получает одну картинку и сохраняет 7 файлов. Это вход в сером цвете и 6 результатов обработки. Четыре демонстрационных входа дают 28 выходных картинок. По умолчанию Visual Studio запускает обработку только одной шумной картинки с фигурами.

| Имя результата | Что это |
|---|---|
| gray.png | Исходная картинка, переведённая в оттенки серого |
| laplace.png | Контуры методом Лапласа |
| roberts.png | Контуры методом Робертса |
| sobel.png | Контуры методом Собела |
| mean3.png и mean5.png | Обычное усреднение с окном 3×3 и 5×5 |
| gauss5.png | Гауссово усреднение, окно 5×5, σ=1 |

## 4. Изображение и пиксели

### 1. Что такое изображение для программы?

Обычная таблица чисел. Каждая ячейка — пиксель. У серого изображения число означает яркость: 0 — чёрный, 255 — белый, между ними — оттенки серого. Чем больше число, тем светлее пиксель.

### 2. Что такое ширина, высота и каналы?

Ширина — сколько пикселей в строке. Высота — сколько строк. Каналы — какие значения хранятся внутри пикселя. Один канал — яркость. Два — яркость и прозрачность. Три — красный, зелёный, синий. Четыре — те же три цвета и прозрачность. Прозрачность также называют альфа-каналом.

**3. Почему мы используем серые картинки?**

По заданию, если цвет не является главным предметом работы, нужны чёрно-белые изображения. Так проще сравнивать изменения яркости. Если подать цветную картинку, программа сама переведёт её в оттенки серого.

**4. Как цвет превращается в серый?**

По формуле: яркость = 0.299×R + 0.587×G + 0.114×B. Потом результат округляется. Зелёный имеет больший вес, потому что зрение сильнее воспринимает его вклад в яркость. Например, чисто красный пиксель (255,0,0) превратится примерно в 76. Готовый серый пиксель не пересчитывается по этой формуле: его яркость уже известна.

**5. Как пиксели лежат в памяти?**

Не как нарисованная таблица, а одной длинной последовательностью: сначала первая строка, потом вторая и так далее. Для поиска нужного пикселя используется формула index=(y×width+x)×channels. Например, ширина 10, три канала, координаты x=3, y=2: index=(2×10+3)×3=69. Его цвета лежат в ячейках 69, 70 и 71. Нумерация начинается с нуля.

**6. Что значит stride в сохранении PNG?**

Это размер одной строки в байтах. В нашем массиве он равен width×channels. Например, строка из 256 пикселей RGB занимает 256×3=768 байтов. Это число нужно библиотеке, чтобы правильно перейти к следующей строке.

**7. Что за тип uint8_t и почему нельзя в нём всё считать?**

Это число от 0 до 255, занимающее один байт. Оно удобно для готовой яркости. Но сумма нескольких пикселей может быть больше 255, а разность — отрицательной. Поэтому промежуточные расчёты идут в int или double, и только готовый результат превращается в байт.

**8. Почему результат ограничивается от 0 до 255?**

Потому что в одном байте нельзя сохранить, например, яркость 700. Функция put сначала делает всё меньше нуля равным нулю, всё больше 255 — равным 255, затем округляет и записывает пиксель. Это ограничение диапазона. Оно не растягивает яркости всей картинки от чёрного до белого.

**5. Память, копирование и умные указатели**

**9. Что такое умный указатель простыми словами?**

Это объект, который следит за выделенной памятью и сам освобождает её, когда она больше не нужна. У нас используется unique_ptr. Он — единственный владелец массива пикселей. Это помогает избежать забытых освобождений памяти и освобождения одного массива два раза.

Короткий ответ: «unique_ptr автоматически освобождает массив изображения. Ручной delete[] в Image не нужен».

**10. Чем unique_ptr отличается от обычного указателя и shared_ptr?**

Обычный указатель только хранит адрес: кто и когда освободит память, нужно продумывать отдельно. unique_ptr владеет памятью один. shared_ptr позволяет нескольким объектам совместно владеть одной памятью. Нам совместное владение не нужно: каждое изображение хранит свои пиксели.

**11. Что такое глубокое копирование?**

У двух изображений разные массивы с одинаковыми начальными данными. Если изменить одну картинку, вторая останется прежней. Для этого наш конструктор копирования выделяет новый массив и копирует туда байты. Если бы просто копировался адрес, оба объекта обращались бы к одним пикселям.

**12. Чем Image b=a отличается от b=a?**

В первом случае b только создаётся — работает конструктор копирования. Во втором b уже существует — работает оператор присваивания. Оба варианта у нас создают независимые данные. В

присваивании есть защита от a=a, чтобы объект не пытался без необходимости копировать сам себя.

**13. Что делает std::move(copy)?**

Передаёт владение временным массивом в m_data, не копируя все пиксели ещё раз. Здесь перемещается именно unique_ptr. Нельзя говорить, что у Image реализованы все операции перемещения: отдельных перемещающих конструктора и присваивания у него нет.

**14. Что произойдёт, если памяти не хватит?**

Выделение памяти выдаст исключение — сообщение об ошибке для вызывающего кода. При присваивании новый массив сначала готовится отдельно. Поэтому, если память не выделилась, старое изображение не портится. В main стандартное исключение перехватывается и печатается сообщение.

**15. Для чего конструктор и деструктор?**

Конструктор готовит объект к работе: задаёт размеры и выделяет память или загружает файл. Деструктор выполняется, когда объект уничтожается. У Image он записан как = default, потому что unique_ptr уже умеет сам освобождать память. Это пример RAII: ресурс освобождается вместе с объектом, которому принадлежит.

**16. Что будет при неверных размерах и пустой картинке?**

Image() без параметров — допустимый пустой объект. У него размеры нулевые, а память не выделена. Но Image(0,1,3) — неправильный запрос на создание картинки, он вызывает ошибку. Также проверяются число каналов 1 –4 и слишком большие размеры. Проверка размеров не означает, что для любого допустимого размера точно хватит памяти.

**17. Что будет при обращении за край изображения?**

getColor и setColor сразу закончат работу: ничего не запишут и не полезут в чужую память. getColor при этом не меняет переданные

переменные. Это защита класса. А фильтры для соседей за краем используют другой приём: берут ближайший крайний пиксель через sample.

**18. Есть ли особенность у серого цвета и прозрачности?**

Да. В getColor у серой картинки яркость возвращается в r, а g и b становятся нулями. Поэтому код отдельно учитывает серый вход. setColor не меняет прозрачность: параметра для неё в заданном интерфейсе нет. При создании картинки с альфа-каналом он сначала равен 255 — полностью непрозрачный.

**6. Как находятся контуры**

**19. Что такое контур?**

Это место, где яркость резко меняется. Например, тёмный фон рядом со светлым квадратом. Программа сравнивает соседние пиксели: если значения сильно различаются, получается заметный отклик. Он часто совпадает с границей объекта, но может возникнуть и от шума.

**20. Что значит производная на изображении?**

Это оценка того, насколько быстро меняется яркость. Вместо обычной производной из математики берутся разности соседних чисел. Первая производная показывает сам перепад, вторая — как меняется этот перепад. Робертс и Собел используют первые разности, Лаплас — вторые.

**21. Что такое окно и ядро?**

Окно — маленький участок вокруг пикселя, например квадрат 3×3. Ядро — таблица коэффициентов для этого участка. Значение каждого соседа умножается на свой коэффициент, затем всё складывается. Полученная сумма помогает вычислить новый пиксель.

**22. Как работает Лаплас?**

Складывает верхнего, нижнего, левого и правого соседей и вычитает четыре значения центрального пикселя. Формула: L=левый+правый+верхний+нижний−4×центр. Потом берётся модуль:

отрицательный знак убирается. На ровном фоне получается ноль, около перепада — заметное число.

Его ядро: [0 1 0; 1 −4 1; 0 1 0]. Каждая часть между точками с запятой — отдельная строка.

**23. Почему у Лапласа бывает двойная граница?**

По разные стороны скачка возникают отклики разных знаков. После взятия модуля оба становятся положительными и видимыми. Поэтому одна граница может выглядеть как две соседние линии. Это связано с методом, а не обязательно с ошибкой в коде.

**24. Как работает Робертс?**

Берёт маленькое окно 2×2. Сравнивает пиксели по двум диагоналям: a=f(x,y)−f(x+1,y+1), b=f(x+1,y)−f(x,y+1). Затем вычисляет sqrt(a²+b²). В коде для этого используется hypot. Метод простой и даёт компактные контуры, но не сглаживает шум.

**25. Как работает Собел?**

Берёт окно 3×3 и считает изменение яркости по двум направлениям. Для этого использует две маски:

Sx=[−1 0 1; −2 0 2; −1 0 1].

Sy=[−1 −2 −1; 0 0 0; 1 2 1].

После получения gx и gy вычисляет sqrt(gx²+gy²). Коэффициенты 2 сильнее учитывают соседей в средней строке или столбце. В маске одновременно есть вычисление разности и небольшое сглаживание поперёк неё.

**26. Почему нельзя сказать: «У Собела контур ярче, значит он лучше»?**

У методов разные коэффициенты и разное усиление. Собел может давать большие числа просто из-за своей маски. Кроме того, всё выше 255 обрезается. Нужно смотреть, правильно ли найдены границы, сколько лишних линий появилось и не потерялись ли детали.

**27. Почему шум тоже становится контуром?**

Шум — это случайные изменения яркости. Оператор видит перепад, но не знает, относится ли он к предмету или к случайной помехе. Поэтому на ровном фоне шумной картинки появляются ложные контуры. Собел немного сглаживает шум, но полностью его не убирает.

**28. Что делаем с соседями за пределами картинки?**

Повторяем ближайший пиксель края. Например, для $x=-1$ берём $x=0$. Это делает sample. Если бы снаружи всегда был чёрный цвет, вдоль светлого края могла бы появиться ложная граница. У нас ровная картинка остаётся ровной даже у края.

**29. Получаются строго чёрно-белые линии? Есть порог?**

Нет. Результат может содержать любые серые значения от 0 до 255. Порог вида «больше 100 — белый, иначе чёрный» не используется. Наши операторы также не являются алгоритмом Канни: у Канни есть дополнительные этапы, которых здесь нет.

**7. Как убирается шум**

**30. Что такое гауссов шум?**

К яркости пикселей прибавляются случайные отклонения. Небольшие отклонения встречаются чаще, большие — реже. Из-за этого ровная область становится зернистой. Нормальное распределение этих отклонений называют гауссовым.

**31. Как работает обычное усреднение?**

Берём все числа в окне, складываем и делим на их количество. В окне $3\times3$ — девять чисел, в $5\times5$ — двадцать пять. Положительные и отрицательные отклонения частично компенсируются. Но вместе с шумом усредняются и полезные детали, поэтому изображение размывается.

**32. Чем гауссов фильтр отличается от обычного среднего?**

У обычного среднего все соседи одинаково важны. У гауссова фильтра ближайшие к центру важнее, дальние — слабее влияют на результат. Это задаётся весами $\exp(-(i^2+j^2)/(2\sigma^2))$. После расчёта веса делятся на их общую сумму, чтобы вместе давать 1. Иначе даже ровная картинка могла бы осветляться или затемняться.

**33. Что такое σ в гауссовом фильтре?**

Параметр, который определяет, насколько сильно учитываются дальние соседи. При небольшом σ основное влияние у центра. При большом веса в том же окне становятся более похожими друг на друга. В нашем основном запуске окно 5×5 и σ=1. Для обычного среднего вызывается applyBoxBlur. Для Гаусса — applyGaussianBlur с положительным σ. Эти функции используют общий внутренний расчёт average; он не вызывается напрямую из Source.cpp.

**34. Почему большое окно не всегда лучше?**

Оно усредняет больше соседей и сильнее приглушает шум, но также сильнее размывает края и стирает тонкие линии. На картинке с крупными фигурами это может помочь, а на мелких деталях — навредить. В нашей работе это видно на втором наборе изображений.

**35. Почему размеры окна нечётные?**

Чтобы был один центральный пиксель и одинаковое количество соседей с каждой стороны. Например, у 5×5 радиус равен 2. В коде разрешены нечётные размеры от 1 до 31. Ограничение 31 выбрано в программе, оно не является общим законом фильтрации.

**36. Где мы добавляли шум и чему равно его σ?**

Шум подготовлен заранее через редактор ImageMagick, который вызывает prepare_images.py. В основном опыте C++-программа читает готовый шумный PNG из редактора. Дополнительный режим --random сам создаёт шум для быстрой проверки; это отдельный тест, не источник таблиц

отчёта. Измеренное стандартное отклонение шума около 17.92 уровней яркости. Это не то же самое, что σ=1 у фильтра: там параметр относится к расстоянию между пикселями.

В ImageMagick использовано -attenuate 0.9. Число 0.9 — настройка редактора, а не измеренное σ шума. Вариант 2 разрешает добавить шум в Photoshop или другом графическом редакторе, поэтому ImageMagick соответствует выбранному заданию.

## 8. Как объяснить результаты

### 37. Зачем две разные картинки?

На первой большие фигуры и много ровного фона — хорошо видно, исчезает ли зернистость. На второй тонкие линии и мелкие элементы — видно, не стирает ли фильтр полезные детали. Если проверять только большие фигуры, можно ошибочно решить, что сильное сглаживание всегда полезно.

### 38. Что такое MSE простыми словами?

Мера того, насколько результат отличается от чистой картинки. Для каждого пикселя берём разницу, возводим в квадрат и потом считаем среднее по изображению. Меньше MSE — ближе к эталону. Квадрат нужен, чтобы ошибки разных знаков не погашали друг друга.

### 39. Что такое PSNR?

Другая запись оценки ошибки, в децибелах. Формула: $PSNR=10\times\log10(255^2/MSE)$. Здесь больше — лучше. Важно помнить: сравниваем результат с чистым эталоном, а не с шумным входом. Эти числа описывают совпадение яркостей, но не заменяют просмотр картинок.

### 40. Какие результаты нужно помнить?

На крупных фигурах шумный вход имеет MSE=321.50. После обычного среднего 3×3 ошибка равна 111.66, после среднего 5×5 — 147.53,

после Гаусса 5×5 — 95.38. Все три метода помогли, но увеличение обычного окна с 3 до 5 ухудшило результат из-за размытия границ.

На мелких деталях шумный вход имеет MSE=321.63. После Гаусса 5×5 ошибка 400.27, а после обычного среднего 5×5 — 873.40. Шум стал слабее, но часть тонких линий исчезла. Из наших усредняющих настроек на обеих сценах меньше ошибся Гаусс 5×5 с σ=1, однако даже он не гарантирует улучшения любого изображения.

**41. Почему шум слабее, а ошибка иногда больше?**

Потому что сравнение учитывает не только шум, но и повреждение настоящих объектов. Фильтр может сделать фон гладким и одновременно стереть тонкую линию. Изображение выглядит спокойнее, но дальше от чистого оригинала.

**42. Где программа считает MSE и PSNR?**

Не в Source.cpp. Эти числа считает Python-скрипт make_report.py из сохранённых картинок. Они записаны в reports/metrics.json и отчёт. C++-часть выполняет фильтрацию и сохраняет PNG. Контурные карты не сравниваются с исходной картинкой по MSE: для них нужна отдельная разметка правильных границ, которой здесь нет.

**9. Ошибки и тесты**

**43. Что такое исключение, try и catch?**

Исключение — способ сообщить, что обычное выполнение продолжить нельзя. Внутри try находится работа программы. Если возникает стандартная ошибка, catch её перехватывает, печатает сообщение и завершает запуск. Это не автоматическое исправление файла: причина ошибки остаётся, пока её не исправить.

**44. Что происходит, если входной файл не найден?**

isRead возвращает false. Конструктор Image с путём к файлу превращает это в runtime_error. Затем main печатает сообщение Cannot read и

возвращает код 1. Нужно проверить путь, имя файла и рабочую папку. При неудачном isRead существующая картинка не стирается.

**45. Что происходит, если результат не записывается?**

isWrite возвращает false. Функция saveImage превращает его в ошибку Cannot save. В runFilters лямбда save вызывает эту функцию для каждого результата. main перехватывает ошибку и завершает запуск. Нужно проверить доступность папки и право записи. Если несколько файлов уже сохранены, они останутся: программа не откатывает всё назад и не удаляет их.

**46. Что значит код завершения 1?**

Это сигнал операционной системе: запуск закончился неудачно. Ноль означает успех. Конкретная причина показывается в сообщении, а не в разных кодах для каждой ошибки. Без аргументов запуска программа берёт готовую картинку test_input/shapes_noisy.png и сохраняет результат в test_output/shapes_noisy. Можно указать два аргумента, чтобы выбрать другие пути. Если передать неправильное количество аргументов, появится справка и код 1. Флаг --random запускает дополнительный случайный тест, --help показывает справку. Если готового входа нет, запуск без аргументов создаст случайный тест автоматически.

**47. Что ещё проверяет программа?**

Размеры и число каналов изображения. Непустой серый вход для фильтров. Нечётный размер окна от 1 до 31. Положительное конечное σ для гауссова фильтра. Отдельно есть проверки доступа к пикселям и результата сохранения. Если создание выходной папки выдаст стандартное исключение, main тоже его перехватит.

**48. Зачем тесты и что значат 18 и 323?**

18 — количество тестовых случаев. 323 — отдельных проверяемых условий внутри них. Есть 12 случаев из задания и 6 дополнительных. Проверяются загрузка и запись, каналы, копирование, безопасность и ручные

расчёты фильтров. Все тесты прошли, но это не означает, что программа проверена на абсолютно всех возможных входах.

**49. Какие тесты нужно уметь объяснить?**

На ровном поле контуры должны быть нулевыми, а фильтры — сохранять яркость. На простой ступеньке можно заранее вручную получить ответ и сравнить с программой. При копировании изменение оригинала не должно менять копию. После сохранения PNG снова читается: размеры и прозрачность должны сохраниться. Проверяется и то, что выход за границы не портит переданные значения.

**10. Пять расчётов, которые стоит понять**

**Пример 1. Ровный фон**

Все пиксели равны 90. У Лапласа: 90+90+90+90−4×90=0. У Робертса разности 90−90 тоже равны нулю. У Собела отрицательные и положительные части маски взаимно погашаются. Обычное среднее и Гаусс дают 90.

**Пример 2. Ступенька от 0 до 40**

Рассматриваем пиксель слева от вертикальной границы. В его окне 3×3 каждая строка — [0,0,40]. Лаплас: правый сосед 40, остальные нужные значения нулевые, ответ 40. Робертс: разности −40 и 40, ответ $\sqrt{40^2+40^2}\approx 56.57$, после округления 57. Собел: gx=40+80+40=160, gy=0, ответ 160. Эти три числа проверяются в наших тестах.

**Пример 3. Один белый пиксель среди восьми чёрных**

В окне 3×3 одна яркость 255 и восемь нулей. Обычное среднее: 255/9≈28.33 → 28. Гауссов фильтр 3×3 при σ=1 даёт около 52, потому что центр весит больше соседей. Такой гауссов расчёт есть в тесте; в основном запуске используется окно 5×5.

**Пример 4. Память изображения**

Серая картинка 256×192 содержит 256×192=49 152 байта пикселей, то есть 48 КиБ. RGB-картинка того же размера — в три раза больше: 147 456

байтов. Это размер массива пикселей, а не PNG-файла: файл сжимается и содержит служебные данные.

**Пример 5. Почему ошибки возводятся в квадрат**

Чистые значения [100,100], результат [90,110]. Ошибки −10 и +10. Их обычное среднее равно нулю, хотя оба пикселя неправильные. Квадраты — 100 и 100, поэтому MSE=100. Так ошибки разных знаков не скрывают друг друга.

**11. Если преподаватель спросит глубже**

**50. Почему нельзя записывать результат прямо в исходную картинку?**

Тогда следующий пиксель мог бы использовать уже изменённого соседа. Итог зависел бы от порядка обхода и не соответствовал формуле для исходного изображения. Поэтому фильтр читает gray и пишет в отдельный result. const Image& в параметре дополнительно запрещает менять вход через эту ссылку.

**51. Зачем в isRead ещё один умный указатель с функцией удаления?**

Память, которую вернул stbi_load, должна освобождаться именно stbi_image_free. Поэтому функция освобождения передаётся в unique_ptr отдельно. Обычный unique_ptr массива использовал бы delete[], а это не тот способ, который требует библиотека. Затем пиксели копируются в наш собственный массив.

**52. Почему библиотека stb есть, но фильтры считаются «нашими»?**

stb читает и записывает формат файла. Она не вычисляет вместо нас Собела или усреднение. Их формулы, обход соседей и запись результатов реализованы в Filters.cpp. Готовый внешний алгоритм фильтрации не вызывается.

**53. Что значит namespace, enum class, auto и лямбда?**

namespace Filters — общая «папка имён» для наших функций. Например, Filters::applySobel означает «функция Собела из группы Filters».

enum class Edge используется внутри Filters.cpp, чтобы общий внутренний расчёт различал три оператора; из Source.cpp вызываются отдельные функции applySobel, applyRoberts и applyLaplacian. auto просит компилятор определить тип из выражения справа. Лямбда save — небольшая локальная функция, чтобы не повторять один и тот же код сохранения и проверки ошибки.

**54. Зачем STB_IMAGE_IMPLEMENTATION и почему их нельзя писать везде?**

Они включают саму реализацию библиотеки в Image.cpp. В других файлах нужны только объявления из заголовков. Если включить реализацию в нескольких cpp, сборщик может получить одинаковые определения функций несколько раз и выдать ошибку.

По похожей причине приложение Lab и тесты Tests собираются отдельно: у каждого своя функция main — своя точка запуска. Если добавить обе в одну программу, сборка не получится.

**55. Что такое линейный и нелинейный фильтр?**

Линейный расчёт — это сумма пикселей с постоянными коэффициентами. Так устроено усреднение до округления. Но взятие модуля, округление и обрезание результата 0–255 уже не являются линейными действиями. Поэтому весь итоговый конвейер не нужно без оговорок называть строго линейным.

**56. А у нас точно математическая свёртка?**

Строго в циклах ядро не переворачивается: это корреляция, которую в задачах обработки картинок часто тоже называют применением свёрточной маски. Для симметричных ядер среднего, Гаусса и Лапласа разницы нет. У Собела переворот меняет знаки gx и gy, но после $\sqrt{gx^2+gy^2}$ результат тот же. Если этого не спрашивают, достаточно объяснения через окно и коэффициенты.

**57. Как быстро работает программа и как её можно ускорить?**

Каждый выходной пиксель просматривает своё окно. Для окна 5×5 это 25 соседей. Значит, чем больше изображение и окно, тем больше работы. Оценка прямого фильтра — $O(W \times H \times k^2)$. Гаусс можно ускорить двумя проходами по строкам и столбцам, обычное среднее — специальной таблицей накопленных сумм. В текущем проекте этих ускорений нет: реализован прямой и понятный расчёт.

**58. Что программа пока не умеет?**

У неё нет графического интерфейса и автоматического показа PNG. Параметры фильтров задаются вызовами в runFilters внутри Source.cpp, а не меню. Прозрачность не накладывается на фон при переводе в серый. Уже записанные файлы не удаляются после последующей ошибки. Все результаты нужно оценивать для выбранных входов, а не объявлять один фильтр лучшим навсегда.

Ещё две тонкости: сам setColor не обрезает неправильный цвет — он переводит его в байт; за допустимый диапазон результатов фильтра отвечает put. И проверка σ не защищает от всех крайних значений компьютерной арифметики: очень маленькое положительное σ может вызвать проблемы в вычислении $\sigma^2$. Для работы и тестов используются обычные значения, в основном $\sigma=1$.

**59. Зачем random в Source.cpp и как его объяснить?**

Это дополнительная тестовая картинка, которую можно получить без поиска файла вручную. Команда --random 73 создаёт градиентный фон, шесть прямоугольников со случайными положениями, размерами и яркостями, а также тонкие штрихи. Размер картинки — 320×224. Затем к копии добавляется гауссов шум со стандартным отклонением 18 и запускаются наши фильтры вариантов 1 и 2.

std::mt19937 — генератор последовательности случайных чисел. seed — начальное число: оно задаёт, с какой последовательности начать.

uniform_int_distribution выбирает целые числа для прямоугольников; normal_distribution — шум, где небольшие отклонения встречаются чаще больших. После добавления шума яркость ограничивается 0–255.

Одинаковый seed в одной сборке даёт тот же тест, поэтому ошибку можно повторить. С другим seed картинка меняется. Стандартная библиотека может реализовать распределения по-разному, поэтому не стоит обещать побайтовое совпадение между разными компиляторами.

Чистая картинка и шумная копия лежат в test_input/generated/<seed>, результаты — в test_output/random_<seed>. В генераторе используется глубокое копирование Image noisy(clean): шум меняет только копию. По умолчанию seed=2026. Если готовая картинка для обычного запуска пропала, программа тоже создаст этот дополнительный тест.

Важно для условия: изображения с шумом для основного отчёта подготовлены редактором ImageMagick. Генератор в C++ — дополнительная проверка и не подменяет это требование. Третьего варианта он не добавляет. В Windows SetConsoleOutputCP(CP_UTF8) настраивает вывод русских сообщений в консоли; фильтрацию эта настройка не меняет.

**12. Короткие ответы, которые удобно запомнить**

«Почему unique_ptr?» — Чтобы у массива был один владелец и память освобождалась автоматически.

«Почему отдельный Filters.cpp?» — По условию нельзя расширять интерфейс Image.

«Что такое контур?» — Резкий перепад яркости между соседними областями.

«Почему появился ложный контур?» — Шум тоже создаёт перепад яркости.

«Почему размылось?» — Фильтр смешал значения объекта и фона в одном окне.

«Какой фильтр лучше?» — Из трёх проверенных усреднений меньше ошибся Гаусс 5×5 с σ=1. Но размер окна и полезность сглаживания зависят от изображения.

«Чем подтверждён результат?» — Сравнением с чистыми эталонами, таблицами ошибок и тестами с заранее известными ответами.

«Что происходит при ошибке?» — Выводится сообщение, запуск завершается с кодом 1; причина автоматически не исправляется.

«Что делать, если предложат доработку?» — Сначала объяснить, что сейчас делает программа, затем предложить изменение. Не говорить, что улучшение уже реализовано, если его нет в коде.